

SANA: What Matters for QA Agents over Massive Data Lakes?

Austin Senna Wijaya, Jiaxiang Liu, Haonan Wang, Eugene Wu

 DAPLab Columbia University

New York, USA

{asw2215,jl6235,hw2983,ew2493}@columbia.edu

ABSTRACT

Exploratory question answering (EQA) over data lakes requires an LLM agent to discover relevant sources, analyze retrieved data, and adapt its actions based on intermediate results. End-to-end accuracy alone cannot distinguish failures in search, planning, data analysis, or the agent’s **Action Policy**: its decisions about what to do next and when to submit an answer. We present SANA (Search Agent Navigation Ablation framework), a diagnostic ablation framework that transforms EQA tasks into runtime profiles containing gold source sequence, sanitized subquestions, and execution records. SANA uses these profiles to construct idealized search, planning, and data-analysis tools, allowing each component to be ablated; the residual gap is diagnostic evidence for policy failures.

To illustrate SANA as a reusable evaluation framework, we adapted two recent EQA benchmarks, LakeQA and KramaBench, and evaluated lightweight and mid-sized agents under fixed prompts, budgets, data lakes, and runtimes. Across both benchmarks, data analysis is a consistent bottleneck while planning is less so. Search is a major limitation in LakeQA’s large data-lake setting, but less so for the smaller-scale KramaBench. SANA thus deconstructs end-to-end task accuracies into a diagnosis of where data-lake agents fail, and allows for systematic comparisons of progress in search, planning, data analysis, and agent design.

VLDB Workshop Reference Format:

Austin Senna Wijaya, Jiaxiang Liu, Haonan Wang, Eugene Wu. SANA: What Matters for QA Agents over Massive Data Lakes?. VLDB 2026 Workshop: Systems for Data-centric Agents with Human-in-the-loop (DASHSys).

VLDB Workshop Artifact Availability:

The source code, data, and/or other artifacts have been made available at <https://github.com/Austin-Senna/exploratory-qa-eval>.

1 INTRODUCTION

Exploratory question answering (EQA) was recently introduced in LakeQA [1] as the problem where an agent must answer questions over facts that are distributed across a large data lake. The agent must iteratively reason about missing evidence that must be searched for, inspect and analyze retrieved data, and use intermediate findings to guide subsequent actions. This models practical analytics over data lakes: data scientists and business analysts can

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, ISSN 2150-8097.

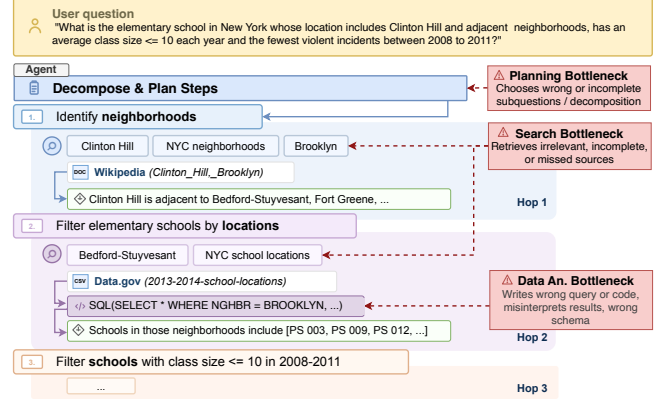


Figure 1: EQA agent runtime loop, with inherent bottlenecks during planning, searching, and data analysis.

formulate high-level questions, but not which tables, metadata records, filters, or computations are needed to answer it.

Evaluating EQA is difficult because success depends not only on the quality of individual capabilities (e.g., search, planning, analysis of retrieved datasets), but also on how the agent composes them over a long trajectory. We refer to the agent’s **Action Policy**, or simply **Policy**, as the turn-level decision process that selects the next action given the question, discovered sources, intermediate observations, tool errors, and remaining budget. At each turn, the policy must decide whether to search, inspect, compute, validate, revise, or submit an answer.

Prior work has advanced individual capabilities needed for EQA, including agentic retrieval [2–4], planning and decomposition [5, 6], tool use [7, 8], and code/SQL generation [9–11]. EQA requires these capabilities to interact in a single long-horizon loop [1]. Recent evaluations study facets of EQA: DCI [4] gives agents direct grep-style corpus access, while Metadata Reasoner [12] evaluates sufficient and minimal dataset selection. Neither isolates the contributions of search, planning, analysis execution, and agent policy to end-to-end accuracy. Consequently, failures from missing data, ignored evidence, incorrect decomposition, and brittle SQL or code remain conflated. SANA provides controlled ablations that diagnose these bottlenecks in end-to-end EQA agents.

To bridge this gap, we propose SANA (Search Agent Navigation Ablation framework), an ablation framework to study EQA agent policies. Specifically, we develop an idealized EQA agent whose trajectory planning, search system, and data analysis code are designed using the ground truth. This produces an upper bound for those components while preserving the agent’s policy decisions, such as invoking tools and submitting its final answer. We then systematically ablate each component with widely used implementations (e.g., BM25 or hybrid for search).

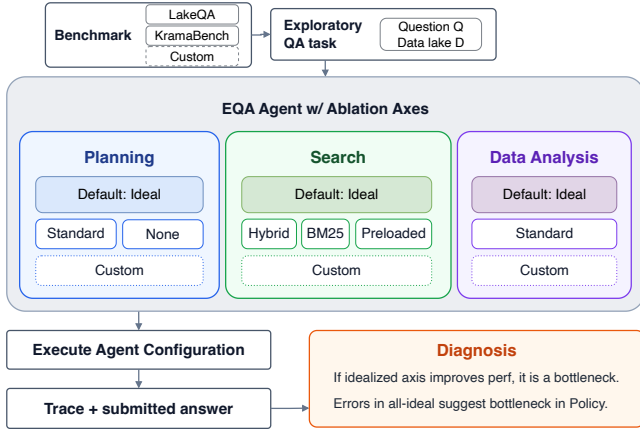


Figure 2: SANA’s three runtime components as an ablation study. Each component can be evaluated with idealized, standard, baseline, or custom implementations.

By idealizing individual components, SANA distinguishes failures caused by planning, search, or data-analysis execution from failures that remain in the agent policy. These residual policy failures include pursuing the wrong source, failing to validate intermediate evidence, submitting an incorrect final answer despite useful evidence, or exhausting the budget without progress. They expose concrete targets for future EQA agents, including stronger evidence tracking, validation, final-answer checking, and stopping criteria.

To summarize, we make the following contributions:

- (1) **SANA**, a diagnostic ablation framework that converts solved EQA tasks into runtime profiles and uses them to construct idealized planning, search, and data-analysis interfaces while preserving the agent policy.
- (2) A **controlled evaluation** on LakeQA [1] and a conversion of KramaBench [13] that isolates the end-to-end contribution of planning, search, and data-analysis execution under fixed agents, prompts, budgets, data lakes, and runtimes.
- (3) An **empirical diagnosis of EQA bottlenecks**: data-analysis execution is a consistent limitation across both benchmarks; search becomes a major limitation in LakeQA’s larger-scale discovery setting; and residual agent-policy failures remain even after component idealization.

2 RELATED WORK

Benchmarks for exploratory QA. QA benchmarks test multi-hop reasoning over heterogeneous data types, including HotpotQA [14], MuSiQue [15], OTT-QA [16], TAT-QA [17], and FeTaQA [18]. These benchmarks stress reasoning over documents, tables, or mixed evidence, but typically operate over a fixed corpus or compact evidence pool. Data-lake benchmarks move closer to exploratory analysis: LakeQA [1] requires agents to search, inspect, compute, and synthesize over large data lakes, while KramaBench [13] evaluates data-to-insight pipelines over multiple structured and unstructured sources. Recent agentic data-discovery evaluations such as DCI [4] and Metadata Reasoner [12] also study agents that interact with a corpus or select task-relevant datasets, but they do not isolate search, planning and execution as separate failure modes.

SANA builds on this benchmark direction, but uses these tasks to diagnose which part of an EQA agent fails.

Component methods for data-lake QA. Prior work improves individual components needed for EQA. Dataset discovery systems such as D3L [19] and Starmie [20] retrieve joinable, unionable, or semantically related tables, while Pneuma [2] and AutoDDG [3] use LLM-generated descriptions to improve tabular retrieval. Text-to-SQL and data-agent benchmarks such as BIRD [11] and Spider 2.0 [10] evaluate data analysis execution once the relevant database context is known. Decomposition methods such as DecompRC [6] and DIN-SQL [5] expose intermediate structure for multi-hop reasoning or SQL generation. SANA treats these as interacting bottlenecks rather than isolated tasks: decomposition must be grounded and guide the agent’s next discovery and analysis, discovery must feed analysis, and analysis must feed later discovery.

Agentic RAG and navigation. Agentic RAG systems use an LLM-controlled policy to decide when to retrieve, call tools, update context, and stop [21], building on reasoning-and-acting methods such as ReAct [22], IRCot [23], and Self-RAG [24]. Recent systems such as MA-RAG [25] and A-RAG [26] further explore multi-agent or hierarchical retrieval architectures. These systems expose the same navigation decisions that arise in EQA, but their evaluations often conflate model policy, retrieval infrastructure, data profiling, tool design, and execution errors. SANA provides a controlled evaluation substrate by holding the task and runtime fixed while ablating search, planning, and execution.

3 THE SANA ABLATION FRAMEWORK

SANA is an ablation framework for search, planning, and execution—the three major components in EQA. This section introduces EQA tasks, why we focus on these components, and the SANA design.

3.1 Anatomy of an Exploratory QA Task

Here, we describe the structure of an EQA task and inter-dependencies between data discovery, question decomposition, data analysis, and tool implementation that are needed to successfully answer a task. In short, data discovery must retrieve the necessary datasets, but if it retrieves too many, then it makes question decomposition challenging. Similarly, if the data analysis tools are poorly implemented or not sufficiently expressive, then the agent cannot extract the necessary information from the datasets to determine answers or next steps.

3.1.1 An EQA Task. In LakeQA [1], an exploratory question-answering (EQA) task is defined as $\tau = (Q, \mathcal{D}, \mathcal{D}_{\text{gold}}, \mathcal{T}, A^*, B)$, where Q is a natural-language question, $\mathcal{D} = \{d_1, \dots, d_N\}$ is a large data lake containing structured and unstructured datasets, $\mathcal{T}$ is the set of tools available to an LLM agent, A^* is the gold answer to Q , $\mathcal{D}_{\text{gold}}$ contains a minimal set of datasets¹ sufficient to derive the gold answer, and the budget B is the maximum number of tool calls permitted.

Generating an answer $\hat{A}$ for an EQA task requires decomposing Q into a sequence of subquestions $(Q_1, Q_2, \dots, Q_K)$, discovering each subquestion’s gold dataset $(d_{\text{gold}}^1, \dots, d_{\text{gold}}^K)$, and answering

¹In practice, $\mathcal{D}_{\text{gold}}$ may not be unique, as it’s currently intractable to enumerate all minimal sets.

$(A_1, A_2, \dots, A_K)$, such that each intermediate answer can be derived from the current subquestion and gold dataset, and the previously derived answers:

$$Q_i, d_{\text{gold}}^i, A_1, \dots, A_{i-1} \models A_i, \quad i = 1, \dots, K. \quad (1)$$

Operationally, an agent must use the tools in $\mathcal{T}$ to discover candidate datasets, analyze them to assess their task-relevance, extract information to compute intermediate answers, and derive the final answer $\hat{A}$ —all within the tool call budget. We next describe the classes of tools in $\mathcal{T}$ and the main challenges posed by EQA. Here, t denotes the t^{th} tool call made by the agent.

3.1.2 Agent tools. The tool set $\mathcal{T}$ supports two classes of operations: (i) dataset search $f(q, k)$, over the data lake and (ii) data analysis $g(c, d)$ over a dataset $d \in \mathcal{D}$. The search tool $f(q, k)$ currently is based on keyword search over dataset contents; it takes as input keywords q , and returns the top- k documents from the data lake. Analysis $g(c, d)$ takes as input an executable program (e.g., Python program, SQL query) c to evaluate over dataset d_i , and returns the corresponding execution I/O traces.

We use $f_t = f_t(q_t, k_t)$ and $g_t = g_t(c_t, d_t)$ as shorthand to denote the specific tool and its arguments in the t^{th} tool call. We denote the cumulative set of discovered datasets at turn t as $\mathcal{D}_{\text{disc}}^t$. In practice, an agent can analyze multiple datasets together, but the text refers to a single dataset for legibility.

3.1.3 Planning. The agent must decompose the EQA question by iteratively identifying the next subquestion to answer given the previous evidence. Formally, given the current discovered datasets $\mathcal{D}_{\text{disc}}^t$ at tool call t , planning produces an ordered sequence of subquestions and evidence choices

$$((\hat{Q}_1, \hat{d}_1), (\hat{Q}_2, \hat{d}_2), \dots)$$

where $\hat{d}_i$ may be unknown at the time $\hat{Q}_i$ is first formulated and must be discovered through subsequent search calls.

3.1.4 Search. The EQA tasks are designed such that the gold evidence $\mathcal{D}_{\text{gold}}$ is necessary in order to correctly answer the subquestions and the task Q . Thus, the agent must, at a minimum, within a budget of B tool calls, identify a superset $\mathcal{D}_{\text{disc}}^B \supseteq \mathcal{D}_{\text{gold}}$. However, high recall alone is not sufficient: $\mathcal{D}_{\text{disc}}^B$ could degenerate to the entire data lake. Search precision is therefore also critical, because irrelevant datasets increase the burden on downstream planning and analysis.

3.1.5 Data analysis. Even if the agent retrieves the gold dataset for the current subquestion, it still needs to analyze that dataset to derive the intermediate answer. In short, the agent must correctly implement the entailment in eq. (1) to derive $\hat{A}_i = g_t(\hat{c}_i, \hat{d}_i)$ without execution failure. If $\hat{A}_i = A_i$ then the agent has correctly answered subquestion i .

3.1.6 Residual Failures. The three challenges above define the major failure classes targeted by SANA’s ablations. However, they do not exhaust all ways an EQA task can fail. For example, even if the agent correctly derived the answers to the correct query decomposition, it may still hallucinate an incorrect final answer $\hat{A} \neq A^*$. They

may also waste the budget by completing irrelevant actions, skipping necessary steps, or hallucinating. We discuss these residual failures in Section 5.3.

3.2 SANA Ablation

SANA isolates the effect of the major components in EQA (section 3.1) by ablating search, planning, and data analysis in isolation or combination. Ablation replaces a component with an idealized version or other implementations. We first describe why ablation cannot naively swap out components and needs to be *intent-based*, then describe the task profile that we use to generate idealized versions for each ablated component. We finally describe the ablation methods for each component and our implementation for each.

3.2.1 Intent-based Ablation. Isolating the contribution of the system components alone requires careful design. For instance, a naive approach to ablate search is to simply give $\mathcal{D}_{\text{gold}}$ to the agent. However, the agent should still be expected to formulate the appropriate search query; we aim to control only the search tool’s quality.

SANA therefore extracts the semantic intention a_t as natural language behind each tool call. For search, SANA extracts a_t as the keyword q ; for execution, SANA extracts a_t as the analysis goal that c seeks to achieve on D .

3.2.2 Task Profiles. SANA constructs task profiles that contain ground-truth information, and uses them to synthesize idealized implementations of each ablated component (Figure 3). A profile is defined by: (Source Sequence) the sequence of gold datasets $\mathcal{D}_{\text{gold}}$ needed for each subquestion, (Sanitized Subquestions) the sequence of subquestions $\hat{Q}$ that have been sanitized to not leak information about the gold dataset, and (Execution Records) that specify the dataset d , the query c , the correct intent for analysis over the dataset a , and the answer A for the subquestions that require data analysis.

EXAMPLE 1. *Figure 3 adapts a LakeQA task. In addition to explicitly listing the order of the three gold datasets, it removes direct mentions of dataset names in each subquestion to avoid leaking search targets. For example, a mention of nypd_crimes is rewritten as “find crime reports reported by NYPD.” Subquestion 1 simply retrieves The Bronx dataset so it does not have an execution record, while the latter steps do have analysis records. The first execution record states the dataset is nypd-complaints, the correct analysis intent is to count the top-offense complaints in 2023, the correct SQL, and the answer 18613.*

In SANA, we adapted tasks from LakeQA and KramaBench. The former contains all of the necessary information for the profile, and we use an LLM to remove identifying signals from each subquestion. The latter only has a question and a monolithic solution script that executes on the task’s dataset, so we use an LLM to decompose this solution script into LakeQA-like sequential, dependent hops such that each hop’s code is execution-verified against the intermediate answer and no subquestion spans more than one source.

3.2.3 Planning Ablation. The planning ablation isolates errors in question decomposition by providing the correct sequence of evidence needs while preserving the agent’s responsibility to formulate search queries, conduct data analysis, and synthesize the final answer. In the standard setting, stronger planners produce

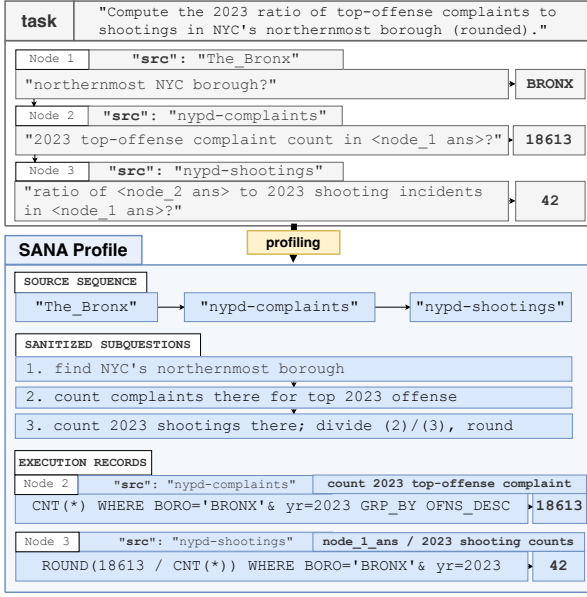


Figure 3: SANA annotates a task into source sequence, sanitized subquestions, and execution records.

sequenced subquestion–dataset pairs closer to the annotated sequence $((Q_1, d_{\text{gold}}^1), \dots, (Q_K, d_{\text{gold}}^K))$.

To generate the idealized component, we provide the agent with $\tilde{Q}$ and ask it to more explicitly state the goal, suggest the tool type to use (e.g., search, query), fallback hints in case a tool call fails. For instance, “count complaints for that borough’s top 2023 offense” would be rewritten as “extract top-count records of the borough in 2023 in the NYC complaints dataset.”

3.2.4 Search Ablation. The search ablation isolates the effect of search precision: it removes irrelevant retrieval results while preserving the agent’s responsibility for deciding what to search for, when to search, and when sufficient evidence has been collected. In the standard setting, the agent searches over the full data lake $\mathcal{D}$, so even when the search tool retrieves gold datasets, it may also return irrelevant datasets that increase downstream planning and analysis burden. To construct the ideal search setting, SANA replaces the standard search tool with an oracle-like operator f^{ideal} whose search space is restricted to $\mathcal{D}_{\text{gold}}$. Thus, every returned dataset is relevant to the task:

$$f^{\text{ideal}}(q, k) \subseteq \mathcal{D}_{\text{gold}} \quad \forall q, k.$$

We implement idealized search by providing the set of gold datasets, the search query, and the search intent to a subagent that selects the datasets that match the search intent. Each dataset is augmented with metadata: an LLM-generated description [2] and a content preview. For a table, we use its schema and row samples; for a document, we use a 100-word preview. In Figure 4, “NYC complaints 2023” would return nypd-complaints, while “NYC farmers market” returns dataset not found.

3.2.5 Data Analysis Ablation. The data analysis ablation isolates implementation errors in executing the intended computation. In the standard setting, given a subquestion–dataset pair (Q_i, d_i) ,

stronger agents are more likely to construct an analysis query that correctly derives the annotated intermediate answer A_i .

SANA replaces the standard analysis tool with $g_t^{\text{ideal}}(c_t, d_t, a_t)$, which contains an additional parameter: the agent’s intent. If a_t matches with the annotated entailment a_i for each (Q_i, d_{gold}^i) pair, SANA returns the verified intermediate answer: $g^{\text{ideal}}(c_t, d_t) = A_i$, otherwise, SANA infers the computational intent from a_t and ensures c_t correctly implements it without execution failure.

We implement idealized data analysis (Figure 4) by first checking if d_t is the gold dataset. If the agent’s intent is the same as the dataset’s analysis intent in the execution record (via a subagent), we return the gold answer. If the intents don’t match or d_t is not a gold dataset, SANA uses a stronger model (gpt-5.4) to generate code that matches the intent and returns the result. If the code execution fails, the model repairs the code and tries again for at most two iterations.

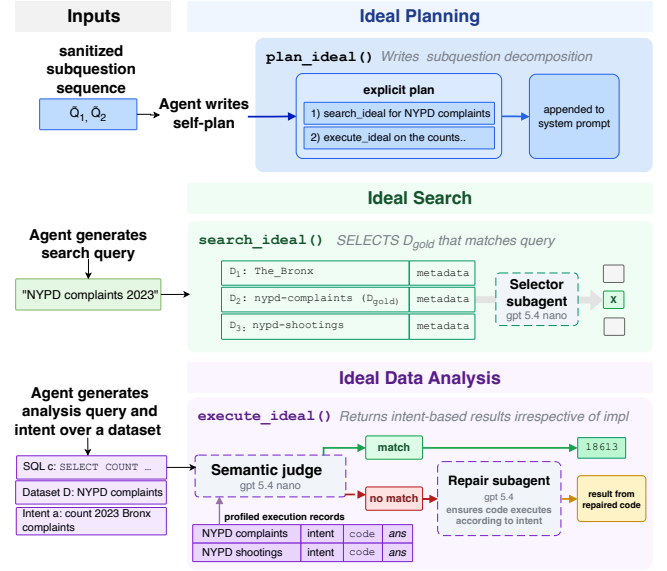


Figure 4: SANA’s ideal ablation consumes one profile input and is modularized as a runtime tool.

4 EVALUATION SETUP

This section describes the task suites, ablation conditions, execution conditions, and metrics used to evaluate SANA. We follow the LakeQA evaluation setting with a budget of $B = 30$ tool calls per run and a maximum runtime of 600 seconds; each run starts from a natural-language question and ends when the agent either calls the answer-submission tool or reaches the run budget. We evaluate two model settings: gpt-5.4-nano as the weaker agent and gpt-5-mini as the stronger agent.

4.1 Benchmarks and Tasks

We evaluate on two task suites. The first is the LakeQA tasks_mini subset [1], containing 135 EQA tasks. The second is the converted KramaBench [13]. To make KramaBench require data discovery, we expose its dataset as a collection of sources [12]; to align with the LakeQA evaluations with $B = 30$ tool calls, we remove the

Kramabench tasks with $|\mathcal{D}_{\text{gold}}| \geq 20$ since these tasks require excessive data discovery relative to the run budget—this reduces the task count from 104 to 83.

Table 1: Benchmark statistics for SANA evaluations. Sources is the number of distinct gold sources across all tasks

Benchmark	Tasks	Sources	Avg. $ \mathcal{D}_{\text{gold}} $	Lake Size $ \mathcal{D} $
LakeQA	135	499	6.9	~40 million
KramaBench-conv.	83	187	2.3	1764

4.2 Ablation Conditions

Targeted Ablations. Each SANA condition is a tuple of planning, search, and data-analysis modes, as summarized in Table 2. The targeted ablations vary one axis at a time while holding the other axes idealized. For example, when ablating planning, search and data analysis are kept idealized such that there are no conflicting bottlenecks.

Table 2: SANA condition space. Each run selects one mode from each axis.

Axis	Mode	Description
Planning	NAIVE	No planning tool; the agent answers directly from the question and available tools.
	STANDARD	Uses the planning tool, but with no sanitized subquestion sequence; the plan is entirely self-written.
	IDEAL	Receives the sanitized subquestion sequence and uses the planning tool to store a plan derived from the reasoning chain.
Search	NAIVE	BM25 sparse lexical search over the data lake.
	STANDARD	Hybrid search with Reciprocal Rank Fusion (RRF), with LLM-generated table descriptions following Pneuma [2] and AutoDDG [3].
	IDEAL	Ideal search tool $f^{\text{ideal}}(q, k)$ over $\mathcal{D}_{\text{gold}}$.
	PRELOADED	$\mathcal{D}_{\text{gold}}$ placed directly in agent context; no search tool.
Data analysis	STANDARD	Writes and runs SQL or Python through the ordinary execution tools.
	IDEAL	Ideal execution tool $g_t^{\text{ideal}}(c_t, d_t, a_t)$ for data analysis.

End-to-end ablations. From table 2, we also define three end-to-end conditions: (i) NAIVE denotes NAIVE Planning, NAIVE Search, and STANDARD data analysis; (ii) STANDARD denotes STANDARD planning, STANDARD search, and STANDARD data analysis; (iii) IDEAL denotes IDEAL planning, IDEAL search, and IDEAL data analysis. This comparison measures the improvement from the lower-bound baseline (NAIVE) to a stronger non-ideal implementation (STANDARD), as well as the remaining gap to the idealized upper bound (IDEAL).

4.3 Execution Conditions

We vary the conditions in section 4.2 while holding the execution environment below fixed.

Execution environment. The agents are given tools for data discovery and data analysis on DuckDB or Python in an isolated sandbox on an AWS g6.2xlarge instance (8 vCPUs, 32 GiB RAM, NVIDIA L4 GPU), where the agent policy is orchestrated with Strands Agent—an open-source model-driven agent SDK that runs the reason-act-observe tool-use loop given a model, system prompt, and tools. Each tool has a 150s timeout.

Baseline difference from LakeQA. All modes use a summarizing conversation manager for context compaction as EQA tasks are often long-winded and have large intermediate query results. We also install a plugin that nudges the agent to reconsider its current strategy when the agent has repeated similar operations over 7 turns. Lastly, search tool returns the sources augmented with meta-data defined in Section 3.2.4. These additions make SANA baseline stronger than the original LakeQA benchmark; thus, our results should not be interpreted as a direct reproduction of LakeQA [1].

4.4 Metrics

Our primary task-success metric is semantic match (SM). We use an LLM-as-a-judge to compare the submitted answer against the gold answer, allowing equivalent answers that differ in formatting, aliases, units, or phrasing.

To measure discovery behavior, let R be the unique sources retrieved from the search tools f_i , and A the unique sources actually accessed by the agent through data-analysis tools g_i . We report:

$$D_{\text{ret}} = |\mathcal{D}_{\text{gold}} \cap R|/|\mathcal{D}_{\text{gold}}|, \quad D_{\text{acc}} = |\mathcal{D}_{\text{gold}} \cap A|/|\mathcal{D}_{\text{gold}}|,$$

the *retrieval recall* and *access recall* over the gold sources. We additionally track tool-call counts and failures in each run’s log.

5 RESULTS & DISCUSSION

This section first reports SANA’s ablation results on LakeQA, then uses LakeQA for ablation and failure analyses. We then evaluate whether the same analysis holds on the converted KramaBench.

5.1 LakeQA Ablation

Table 3: LakeQA ablation matrix (135 tasks/cell).

Model	Plan	Search	Data An.	SM (%)	D_{acc} (%)	D_{ret} (%)	Ret Tool Call	Acc Tool Call
gpt-5.4-nano	NAIVE	IDEAL	IDEAL	34.1	46.5	56.5	3.2	11.6
	STANDARD	IDEAL	IDEAL	31.1	47.0	57.6	4.1	13.2
	IDEAL	NAIVE	IDEAL	23.0	39.0	50.8	5.8	12.3
	IDEAL	STANDARD	IDEAL	26.7	43.4	55.4	4.7	13.2
	IDEAL	IDEAL	STANDARD	28.9	44.0	56.9	3.8	13.6
	IDEAL	IDEAL	IDEAL	37.0	47.1	58.1	4.1	12.5
	IDEAL	PRELOADED	IDEAL	51.8	59.2	—	0.0	17.1
	PRELOADED	IDEAL	IDEAL	66.7	55.1	59.0	5.8	13.1
gpt-5-mini	NAIVE	IDEAL	IDEAL	66.7	56.2	58.8	5.8	14.4
	STANDARD	IDEAL	IDEAL	66.7	56.2	58.8	5.8	14.4
	IDEAL	NAIVE	IDEAL	63.0	50.7	56.5	5.9	15.4
	IDEAL	STANDARD	IDEAL	61.5	52.4	58.0	5.8	15.9
	IDEAL	IDEAL	STANDARD	57.8	53.8	58.7	5.7	12.9
	IDEAL	IDEAL	IDEAL	76.3	56.5	60.2	5.9	14.1
	IDEAL	PRELOADED	IDEAL	77.0	61.7	—	0.0	16.8
	PRELOADED	IDEAL	IDEAL	66.7	55.1	59.0	5.8	13.1

5.1.1 Ablation Delta. Figure 5 reports LakeQA ablation semantic-match deltas. Data discovery is a major bottleneck: Ideal search improves over BM25 by +14.1% for gpt-5.4-nano and +13.3% for gpt-5-mini. The BM25–Pneuma comparison is mixed: Pneuma helps gpt-5.4-nano (+3.7%) but hurts gpt-5-mini (-1.5%), suggesting that non-ideal search improvements remain unreliable. Preloaded Sources is the strongest SANA intervention, where relative to BM25, this yields a +28.9% gain for gpt-5.4-nano and +14.1% for gpt-5-mini. This suggests that weaker models are particularly bottlenecked by search navigation.

Ideal data analysis also yields large gains, especially for gpt-5-mini (+18.5% versus +8.2% for gpt-5.4-nano). This suggests that gpt-5-mini more often reaches the right computation intent but fails during implementation or execution, while gpt-5.4-nano more often fails by requesting the wrong operations.

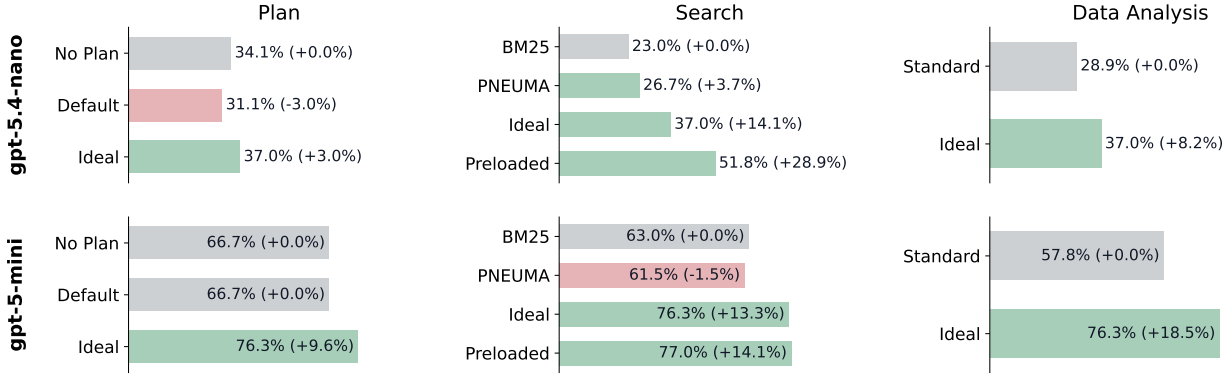


Figure 5: LakeQA ablation semantic-match delta.

Planning gains are smaller. Ideal planning improves gpt-5.4-nano by +3.0% and gpt-5-mini by +9.6%, suggesting that reasoning chains scale with the model’s capability, discussed in Section 5.2.1. Standard planning gives almost no gain, indicating that self-written plans can introduce unnecessary steps or structure that do not necessarily prevent drift or wrong-source navigation.

Table 4: LakeQA end-to-end mode comparison ($n = 135$).

Model	Mode	SM (%)	D_{ret} (%)	D_{acc} (%)	Ret Tool Call	Acc Tool Call
gpt-5.4-nano	NAIVE	20.7	45.4	30.7	5.0	11.0
	STANDARD	19.3 (-1.5)	53.2 (+7.8)	39.5 (+8.8)	5.2	14.1
	IDEAL	37.0 (+16.3)	58.1 (+12.7)	47.1 (+16.4)	4.1	12.5
gpt-5-mini	NAIVE	56.3	53.8	45.5	7.2	12.1
	STANDARD	57.8 (+1.5)	56.9 (+3.1)	48.5 (+3.0)	6.0	13.2
	IDEAL	76.3 (+20.0)	60.2 (+6.4)	56.5 (+11.0)	5.9	14.1

5.1.2 End-to-End Mode Comparison. Table 4 compares the three end-to-end modes: Naive, Standard, and Ideal. Standard provides almost no gain over Naive, giving gpt-5-mini +1.5% and giving gpt-5.4-nano -1.5% even when D_{ret} and D_{acc} are both improved. In contrast, Ideal substantially improves both models, raising gpt-5.4-nano from 20.7% to 37.0% SM and gpt-5-mini from 56.3% to 76.3% SM. This gap shows that stronger non-ideal components do not fully remove the main EQA bottlenecks even when more data is accessed and retrieved.

5.2 LakeQA Ablation Analysis

5.2.1 Planning Ablation. Planning Idealization produces smaller gains than search or data-analysis Idealization on LakeQA. To understand why, we separate two questions: whether the agent can write a reasonable plan, and whether it actually follows that plan during the trajectory.

Plan Similarity. We first compare the plans produced under Standard and Ideal planning. A gpt-5.4-mini judge labels a Standard plan as similar to the Ideal plan when it preserves the same main subquestions, source path, dependency order, and core computation. By this measure, Standard plans are often close to the Ideal scaffold: 81.5% of gpt-5.4-nano plans and 77.8% of gpt-5-mini plans are marked similar to their corresponding Ideal plans. This suggests that agents can already create plans covering the right subquestions and intents.

Plan Trajectory. To check whether the agent actually follows their plan, we audit each trajectory against the sanitized subquestion sequence. A gpt-5.4-mini judge analyzes each run’s agent tool-call trajectory, labelling a trajectory as FOLLOWED when the agent completes the intended ordered subproblems and MOSTLY FOLLOWED when the trajectory preserves the main structure but skips or detours around a minor step.

Table 5: LakeQA plan trajectory audit over targeted plan ablations.

Agent	Plan	Followed	Followed+Mostly
gpt-5-mini	Naive	21.2%	49.6%
	Standard	12.6%	46.4% (-3.2)
	Ideal	25.9%	56.5% (+6.9)
gpt-5.4-nano	Naive	5.7%	19.0%
	Standard	8.4%	24.0% (+4.9)
	Ideal	7.9%	28.1% (+9.1)

The trajectory audit clarifies why ideal planning has limited impact. gpt-5.4-nano is especially poor at following plans: only 19.0% of trajectories are at least MOSTLY FOLLOWED at baseline, and ideal only raises it to 28.1%. gpt-5-mini follows plans more often, but the gain from ideal planning is still modest (49.6% to 56.5%). Thus, simply providing the correct decomposition does not mean agents follow that decomposition reliably. These results suggest that agents are more bottlenecked by their ability to follow plans than their ability to decompose questions; stronger runtime scaffolds for tracking progress and recovering when their trajectory drifts from the plan are needed.

5.2.2 Search Ablation. Figure 5 separates two search-related bottlenecks: retrieval quality and search navigation. Ideal search improves over BM25 for both models, showing that retrieving the right sources from a large data lake is a major bottleneck. However, Preloaded Sources improves over Ideal Search substantially on gpt-5.4-nano while barely affecting gpt-5-mini. This suggests that weaker agents also struggle at search navigation: formulating the right search queries, deciding when to search, and continuing using the right sources once they are available. In contrast, the stronger model gains little improvement from removing search entirely.

Search ablation therefore exposes two separate needs: better retrieval infrastructure and better agent policies for query formulation, source commitment, and evidence collection.

Table 6: Failure families in LakeQA traces. Entries are event shares; Meaning lists (gpt-5-mini, gpt-5.4-nano) subgroups.

Group	gpt-5-mini	gpt-5.4-nano	Meaning
Task/planning failures	7.4%	11.3%	Reasoning chain divergence (5.4%, 8.7%); question constraint misread (1.9%, 2.6%).
Wrong source target failures	0.0%	7.6%	Chose the wrong dataset, table, source family, or source version.
Execution/computation failures	39.6%	26.2%	Computation intent failures: wrong scope/filter (30.9%, 20.1%); computation/aggregation error (4.9%, 2.8%); extraction/parsing error (3.9%, 3.2%).
Incomplete evidence failures	12.2%	12.3%	Ran out of budget before gathering enough evidence (10.3%, 5.0%); submitted early with incomplete evidence (1.9%, 7.3%).
Turn-waste failures	2.1%	8.8%	Repeated actions without progress: query execution/repair loop (0.8%, 5.5%); schema inspection loop (0.2%, 1.8%); low-yield search loop (0.6%, 1.4%); same-hop repetition even after gathering enough evidence (0.6%, 0.2%).
Finalization failures	21.0%	13.2%	Correct evidence or computed value appeared, but the submitted answer was wrong.
Tool blocker failures	17.7%	20.6%	Files, tools, repair calls, unsupported formats, or runtime limits blocked progress.

5.2.3 Data Analysis Ablation. Ideal data analysis helps gpt-5-mini more than gpt-5.4-nano on LakeQA (+18.5% vs. +8.2% in Figure 5). This suggests that gpt-5-mini often reaches the right source context and computation intent, but fails to implement the operation correctly in SQL or Python. The smaller gain for gpt-5.4-nano does not mean that execution is less important for weaker models: rather, many gpt-5.4-nano failures occur before data analysis implementation becomes a bottleneck: choosing the wrong source, scope, or analysis intent. Since ideal execution preserves the agent’s stated intent, it fixes implementation errors but not wrong requests. Thus, data analysis ablation mainly separates code-generation failures from upstream source-selection and intent errors.

5.3 LakeQA Failure Analysis

In this section, we audit failed traces and categorize where every evaluation run with an incorrect semantic outcome fails.

Audit Method. Each failed run is reviewed using a two-stage audit with gpt-5.4-mini. First, the auditor reads the task, the SANA profile, evaluation results, and log trace, then extracts a compact evidence trace of all failures and the turns at which they occur. Then, a separate labeler assigns one or more failure events from the taxonomy in Table 6 using only that evidence trace, such that a single failed run may contribute multiple failures such as answer finalization and planning failures. Since co-occurring symptoms are recorded together regardless of root cause (e.g., misreading question constraint and then filtering the wrong rows returns both constraint misread and wrong scope/filter labels), we treat these LLM-assisted labels as diagnostic rather than ground truth.

Figure 6 and Table 6 explain why the two models respond differently to the same conditions. Gpt-5.4-nano produced 1018 failure events over 586 failed runs and gpt-5-mini 515 over 297, about 1.74 events per failed run for both models. Table 6 gives the event distribution within each model and Figure 6 the condition-level counts. The distributions differ: gpt-5.4-nano’s failures spread across all groups, while gpt-5-mini’s concentrate in execution/computation (57.2% of its failed runs) and finalization (35.7%).

The clearest gap is source scoping: gpt-5-mini has 0% wrong-source failure labels whereas gpt-5.4-nano receives 12.8%. This matches fig. 5, where gpt-5.4-nano benefits more from Ideal Search

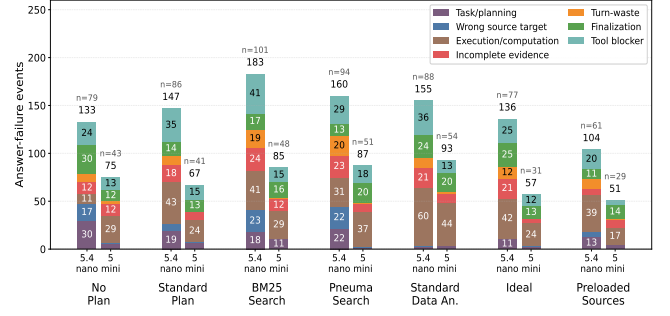


Figure 6: Answer-failure groups across LakeQA ablations. Bold numbers are total events and n the failed runs per bar.

and Preloaded Sources as they prevent wrong-source branches. Gpt-5.4-nano’s other disproportionate failures—turn-waste (8.8% gpt-5.4-nano vs. 2.1% gpt-5-mini), reasoning-chain divergence (8.7% vs. 5.4%), and early submission with incomplete evidence (7.3% vs. 1.9%)—are trajectory failures, agreeing with Subsection 5.2.1 where gpt-5.4-nano is worse at staying on its plan.

Idealizing search (BM25 to Ideal in Figure 6) reduces gpt-5.4-nano’s wrong-source (23 to 0), turn-waste (19 to 12), and divergence (18 to 11) events, partly because failed runs themselves drop (101 to 77). For gpt-5-mini these are already near zero and idealized search mainly trims incomplete-evidence errors, suggesting the stronger model already selects sources and operations well while the weaker one still struggles.

Planning is more selective, helping mainly the smaller model read question constraints and finalize answers. Finalization, computation intent, and turn budget remain failure modes for both, even under Ideal and Preloaded where source access is solved. Overall the two models fail differently: gpt-5-mini usually reaches the right evidence but mis-executes or mis-finalizes, while gpt-5.4-nano’s failures spread across source targeting, plan divergence, turn-waste, and premature stopping, reflecting weaker trajectory control. The ablation gains thus reflect differences in action policies, not only component quality.

5.4 KramaBench Ablation

Table 7: KramaBench ablation matrix (83 tasks/cell).

Model	Plan	Search	Data An.	SM (%)	D_{acc} (%)	D_{ret} (%)	Ret Tool Call	Acc Tool Call
gpt-5.4-nano	NAIVE	IDEAL	IDEAL	75.9	81.7	89.9	1.7	3.2
	STANDARD	IDEAL	IDEAL	85.5	89.3	91.0	1.4	3.7
	IDEAL	NAIVE	IDEAL	85.5	50.1	78.9	2.9	4.0
	IDEAL	STANDARD	IDEAL	90.4	43.5	77.1	2.1	4.0
	IDEAL	IDEAL	STANDARD	63.9	92.9	93.9	1.3	2.9
	IDEAL	IDEAL	IDEAL	87.9	92.3	95.8	1.6	3.5
gpt-5-mini	IDEAL	PRELOADED	IDEAL	91.6	98.4	–	0.0	3.7
	NAIVE	IDEAL	IDEAL	86.8	93.9	95.8	1.4	4.6
	STANDARD	IDEAL	IDEAL	89.2	93.7	94.3	1.4	4.6
	IDEAL	NAIVE	IDEAL	84.3	51.6	82.0	2.7	4.2
	IDEAL	STANDARD	IDEAL	90.4	50.8	78.9	2.0	4.1
	IDEAL	IDEAL	STANDARD	75.9	96.7	96.7	1.5	3.4
	IDEAL	IDEAL	IDEAL	90.4	97.0	97.6	1.2	4.1
	IDEAL	PRELOADED	IDEAL	92.8	99.0	–	0.0	4.6

5.4.1 Ablation Delta. Figure 7 shows a different profile from LakeQA. Data analysis is the largest KramaBench intervention, improving gpt-5.4-nano by +24.1% and gpt-5-mini by +14.5%, indicating that KramaBench is analysis-heavy. Search is less limiting: Ideal search

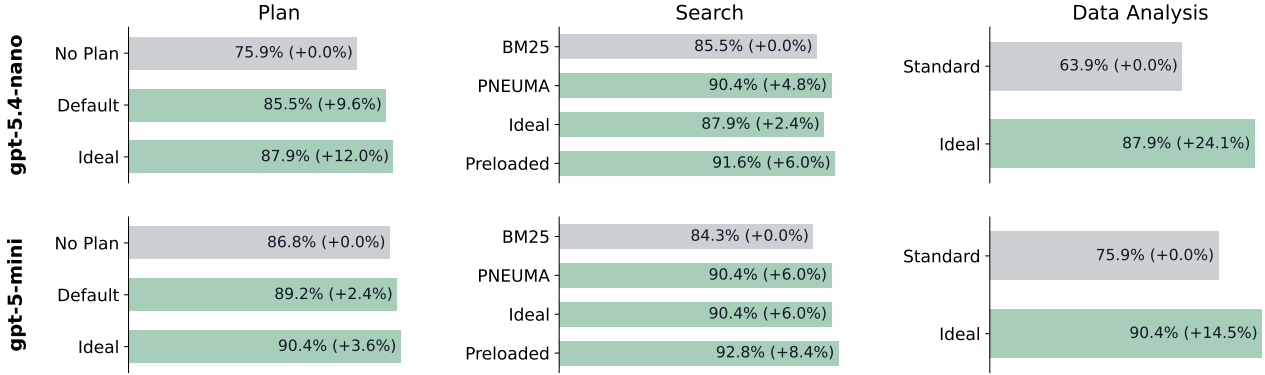


Figure 7: KramaBench ablation semantic-match delta.

improves over BM25 by +2.4% for gpt-5.4-nano and +6.0% for gpt-5-mini, while Preloaded Sources adds +6.0% and +8.4%. This likely reflects the benchmark construction: retrieval is artificially introduced by converting KramaBench into a LakeQA-style task, while the underlying tasks remain centered on data-to-insight execution. Planning helps gpt-5.4-nano more than gpt-5-mini, suggesting that weaker models benefit more from explicit decomposition on KramaBench.

Table 8: KramaBench end-to-end mode comparison ($n = 83$).

Model	Mode	SM (%)	D_{ret} (%)	D_{acc} (%)	Ret Tool Call	Acc Tool Call
gpt-5.4-nano	NAIVE	44.6	64.2	31.8	3.2	3.3
	STANDARD	57.8 (+13.3)	70.6 (+6.4)	44.6 (+12.8)	2.4	3.6
	IDEAL	87.9 (+43.4)	95.8 (+31.6)	92.3 (+60.4)	1.6	3.5
gpt-5-mini	NAIVE	62.6	64.7	36.0	3.8	3.6
	STANDARD	66.3 (+3.6)	68.5 (+3.9)	34.8 (-1.2)	3.2	3.8
	IDEAL	90.4 (+27.7)	97.6 (+32.9)	97.0 (+61.1)	1.2	4.1

5.4.2 End-to-End Mode Comparison. Table 8 shows that Standard improves over Naive on KramaBench, especially for gpt-5.4-nano (+13.3%). Ideal produces the largest gains, reaching 87.9% SM for nano and 90.4% SM for gpt-5-mini. Under Ideal, retrieval recall reaches 95.8%–97.6% and access recall reaches 92.3%–97.0%, so the remaining errors are unlikely to come mainly from missing sources.

6 CONCLUSION AND LIMITATIONS

Conclusion. SANA asks a diagnostic question: when an EQA agent fails over a data lake, which part of the runtime is responsible? We answer this by varying only the planning, search, and data-analysis interfaces. Across LakeQA and the converted KramaBench subset, the results show that EQA failures are not explained by a single component. Data analysis is a consistent bottleneck, search becomes a major limitation in large-scale discovery settings, and residual policy failures remain even after component idealization.

On LakeQA, search is a major bottleneck. Ideal search improves over BM25 for both agents, and preloading gold sources further improves the weaker agent, showing that LakeQA stresses both retrieval quality and search navigation: formulating useful queries, deciding when to search, and committing to the right source context. Data analysis is also critical because agents that reach relevant sources can still fail to implement its intent. Planning is less limiting:

agents often produce plans close to the gold decomposition, but they do not reliably follow those plans.

The converted KramaBench has a different bottleneck. Because its tasks contain fewer gold sources and remain closer to data-to-insight analysis, search is less dominant, while data-analysis becomes the main bottleneck. This shows that SANA does not rank search, planning, or execution as universally hardest, but exposes how bottlenecks change with the benchmark structure.

On LakeQA, the failure audit helps explain the residual gap left after component idealization. Smaller agents fail broadly across source choice, progress control, incomplete evidence, tool blockers, and finalization. Stronger agents are better at staying within the right source context, but still fail during computation and final-answer synthesis. These results suggest that future EQA systems need more than better retrievers or stronger code generation. They need runtime policies that track subgoal progress, commit to validated sources, recover from trajectory drifts, verify intermediate evidence, and check the final answer before stopping.

Limitations. (i). **Diagnostic idealization.** SANA’s ideal tools are diagnostic approximations of the oracles. Ideal search selects only from $\mathcal{D}_{gold}$, so alternative valid sources outside the profile are not credited. Similarly, ideal data analysis relies on semantically matching against annotated records; false matches or missed matches may happen. Thus, SANA’s ablations should be interpreted as approximate diagnostic upper bounds. (ii). **Task-suite scope.** The results are tied to the evaluated task suites. We evaluate LakeQA and a LakeQA-converted KramaBench. The conclusions should therefore be read as evidence about these data-lake settings, not as a universal ranking of search, planning, and data analysis difficulty across all agent benchmarks. (iii). **Judge-based measurements.** Several measurements rely on LLM judges, including semantic-match, plan-trajectory assessment, and failure-audit labeling. These judges make the evaluation practical and allow semantically equivalent answers to be credited, but they can introduce false positives, false negatives, or taxonomy-dependent failure assignments.

REFERENCES

- [1] Haonan Wang, Jiaxiang Liu, Yurong Liu, Austin Senna Wijaya, Tianle Zhou, Eden Wu, Yijia Chen, Wanting You, Reya Vir, Daniela Pinto Veizaga, Grace Fan, Yusen Zhang, Juliana Freire, and Eugene Wu. 2026. LAKEQA: An Exploratory QA Benchmark over a Million-Scale Data Lake. Preprint. <https://www.eugenewu.net/>
- [2] Muhammad Imam Luthfi Balaka, David Alexander, Qiming Wang, Yue Gong, Adila Krisnadhi, and Raul Castro Fernandez. 2025. Pneuma: Leveraging llms for tabular data representation and retrieval in an end-to-end system. *Proceedings of the ACM on Management of Data* 3, 3 (2025), 1–28.
- [3] Haoxiang Zhang, Yurong Liu, Aécio Santos, Wei-Lun Hung, and Juliana Freire. 2026. Autoddg: Automated dataset description generation using large language models. *Proceedings of the ACM on Management of Data* 4, 1 (SIGMOD (2026), 1–27.
- [4] Zhuofeng Li, Haoxiang Zhang, Cong Wei, Pan Lu, Ping Nie, Yi Lu, Yuyang Bai, Shangbin Feng, Hangxiao Zhu, Ming Zhong, Yuyu Zhang, Jianwen Xie, Yejin Choi, James Zou, Jiawei Han, Wenhui Chen, Jimmy Lin, Dongfu Jiang, and Yu Zhang. 2026. Beyond Semantic Similarity: Rethinking Retrieval for Agentic Search via Direct Corpus Interaction. arXiv:2605.05242 [cs.IR] <https://arxiv.org/abs/2605.05242>
- [5] Mohammadreza Pourreza and Davood Rafiei. 2023. DIN-SQL: Decomposed In-Context Learning of Text-to-SQL with Self-Correction. arXiv preprint arXiv:2304.11015. <https://arxiv.org/abs/2304.11015>
- [6] Sewon Min, Victor Zhong, Luke Zettlemoyer, and Hannaneh Hajishirzi. 2019. Multi-hop reading comprehension through question decomposition and rescoring. In *Proceedings of the 57th annual meeting of the association for computational linguistics*. 6097–6109.
- [7] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. 2024. Toolllm: Facilitating large language models to master 16000+ real-world apis. In *International Conference on Learning Representations*, Vol. 2024. 9695–9717.
- [8] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools. *Advances in neural information processing systems* 36 (2023), 68539–68551.
- [9] Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, et al. 2018. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-sql task. In *Proceedings of the 2018 conference on empirical methods in natural language processing*. 3911–3921.
- [10] Fangyu Lei, Jixuan Chen, Yuxiao Ye, Ruisheng Cao, Dongchan Shin, Hongjin Su, Zhaoqing Suo, Hongcheng Gao, Wenjing Hu, Pengcheng Yin, et al. 2025. Spider 2.0: Evaluating language models on real-world enterprise text-to-sql workflows. In *International Conference on Learning Representations*, Vol. 2025. 28691–28735.
- [11] Jinyang Li, Binyuan Hui, Ge Qu, Jiayi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Ruiying Geng, Nan Huo, et al. 2023. Can llm already serve as a database interface? a big bench for large-scale database grounded text-to-sqls. *Advances in Neural Information Processing Systems* 36 (2023), 42330–42357.
- [12] Jiani Zhang, Serkan O. Arik, Cosmin Arad, Fatma Ozcan, and Alon Halevy. 2026. An Agentic Approach to Metadata Reasoning. arXiv:2604.20144 [cs.DB] <https://arxiv.org/abs/2604.20144>
- [13] Eugenie Lai, Gerardo Vitagliano, Ziyu Zhang, Om Chabra, Sivaprasad Sudhir, Anna Zeng, Anton A Zabreyko, Chenning Li, Ferdi Kossmann, Jialin Ding, et al. 2025. Kramabench: A benchmark for ai systems on data-to-insight pipelines over data lakes. arXiv preprint arXiv:2506.06541 (2025).
- [14] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William Cohen, Ruslan Salakhutdinov, and Christopher D Manning. 2018. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 conference on empirical methods in natural language processing*. 2369–2380.
- [15] Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. 2022. MuSiQue: Multihop Questions via Single-hop Question Composition. *Transactions of the Association for Computational Linguistics* 10 (2022), 539–554.
- [16] Wenhui Chen, Ming-Wei Chang, Eva Schlinger, William Wang, and William W Cohen. 2020. Open question answering over tables and text. arXiv preprint arXiv:2010.10439 (2020).
- [17] Fengbin Zhu, Wenqiang Lei, Youcheng Huang, Chao Wang, Shuo Zhang, Jiancheng Lv, Fuli Feng, and Tat-Seng Chua. 2021. TAT-QA: A Question Answering Benchmark on a Hybrid of Tabular and Textual Content in Finance. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, Chengqing Zong, Fei Xia, Wenjie Li, and Roberto Navigli (Eds.). Association for Computational Linguistics, Online, 3277–3287. <https://doi.org/10.18653/v1/2021.acl-long.254>
- [18] Linyong Nan, Chiachun Hsieh, Ziming Mao, Xi Victoria Lin, Neha Verma, Rui Zhang, Wojciech Kryściński, Hailey Schoelkopf, Riley Kong, Xiangru Tang, et al. 2022. FeTaQA: Free-form table question answering. *Transactions of the Association for Computational Linguistics* 10 (2022), 35–49.
- [19] Alex Bogatu, Alvaro AA Fernandes, Norman W Paton, and Nikolaos Konstantinou. 2020. Dataset discovery in data lakes. In *2020 IEEE 36th International Conference on Data Engineering (ICDE)*. IEEE, 709–720.
- [20] Grace Fan, Jin Wang, Yuliang Li, Dan Zhang, and Renée Miller. 2022. Semantics-aware dataset discovery from data lakes with contextualized column-based representation learning. arXiv preprint arXiv:2210.01922 (2022).
- [21] Aditi Singh, Abul Ehtesham, Saket Kumar, Tala Talaie Khoei, and Athanasios V Vasilakos. 2025. Agentic retrieval-augmented generation: A survey on agentic rag. arXiv preprint arXiv:2501.09136 (2025).
- [22] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2022. React: Synergizing reasoning and acting in language models. arXiv preprint arXiv:2210.03629 (2022).
- [23] Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. 2023. Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions. In *Proceedings of the 61st annual meeting of the association for computational linguistics (volume 1: long papers)*. 10014–10037.
- [24] Akari Asai, Zeqiu Wu, Yizhong Wang, Avi Sil, and Hannaneh Hajishirzi. 2024. Self-rag: Learning to retrieve, generate, and critique through self-reflection. In *International conference on learning representations*, Vol. 2024. 9112–9141.
- [25] Thang Nguyen, Peter Chin, and Yu-Wing Tai. 2025. Ma-rag: Multi-agent retrieval-augmented generation via collaborative chain-of-thought reasoning. arXiv preprint arXiv:2505.20096 (2025).
- [26] Mingxuan Du, Benfeng Xu, Chiwei Zhu, Shaohan Wang, Pengyu Wang, Xiaorui Wang, and Zhendong Mao. 2026. A-RAG: Scaling Agentic Retrieval-Augmented Generation via Hierarchical Retrieval Interfaces. arXiv preprint arXiv:2602.03442 (2026).